

Tool for Runtime Monitoring of DiSTL on Distributed Cyber-Physical Systems

Charles Koll¹[0000–0001–5941–250X]

Oregon State University, Corvallis OR 97331, USA
kollch@oregonstate.edu

1 Introduction

This tool performs continuous monitoring of distributed Cyber-Physical Systems (CPS) against a dense-time temporal logic specification. We consider distributed CPS to be multiple coordinating agents, each with a local clock that may drift from the others. These drifting clocks prevent the agents, or even a monitor, from identifying a clear global time. However, use of tools such as NTP [2] allows for a bounding on clock drift – we can say that each agent is at most some constant ε away from global time, i.e. $|c_n(\chi) - \chi| \leq \varepsilon$ where χ is a theoretical global time, c_n is the drift of agent n 's clock, and ε is a fixed constant > 0 . We call this *partial synchrony*.

When monitoring the system's output signal against a formula spec, this lack of synchrony in the system prevents a standard interpretation of the temporal constraints of the formula, blurring the notion of satisfaction. Yet the specification designers wish to specify their constraints via a global clock, abstracting away from the system's asynchrony.

We allow for this abstraction of specification by identifying *all* allowable system states that could potentially satisfy a formula written in Signal Temporal Logic (STL) [1]. In doing so, the monitor can be assured that any system state not returned by our tool definitively does not satisfy the formula. To prevent this problem from being intractable, we restrict the input formula to the monitor to be a significant fragment of STL we label DiSTL, and return an overapproximation of satisfying states. That is, the tool may return a state which it labels as satisfying, even though it does not. However, the converse is not true – any unidentified state will not be satisfiable.

Some system states returned by the monitor are not allowable per our restriction of partial synchrony – this is not a problem, as a given state is easy to check (linear in the number of agents N) by confirming that all values in the state are at most 2ε away from each other.

2 Tool Location

The tool can be found at
<https://github.com/sabotagelab/distl-monitor-and-experiments>.

3 STL and DiSTL

We next provide specifics of STL [1], a common logic for CPS formal requirements, and its fragment DiSTL.

Say that we have a system of N agents, where the output signal of agent n is a function $x_n : \mathbb{R}_{\geq 0} \mapsto \mathbb{R}$. Let **AP** be a set of *atomic propositions*. To every $p \in \text{AP}$ is associated an N -ary function $f_p : \mathbb{R}^N \mapsto \mathbb{R}$. The syntax for STL is given by:

$$\varphi := \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \mathcal{U}_{[a,b]} \varphi$$

STL combines the usual boolean constant True $\top$ and boolean operators (negation $\neg$ and conjunction $\wedge$) with the temporal Until operator $\varphi \mathcal{U}_{[a,b]} \psi$, which means that φ remains true at least until a moment in $[a, b]$ at which ψ becomes true. Formally, let a *trace* $\sigma := (x_1, \dots, x_N)$ be a vector of N continuous-time *synchronous* output signals, one per agent. We think of σ as a regular N -dimensional output signal whose values are measured against global time. The satisfaction of formula φ by trace σ at time t , written $(\sigma, t) \models \varphi$, is defined by:

$$\begin{aligned} (\sigma, t) &\models \top \\ (\sigma, t) &\models p \quad \text{iff } f_p(x_1(t), \dots, x_N(t)) \geq 0 \\ (\sigma, t) &\models \neg\varphi \quad \text{iff } (\sigma, t) \not\models \varphi \\ (\sigma, t) &\models \varphi \wedge \psi \quad \text{iff } (\sigma, t) \models \varphi \text{ and } (\sigma, t) \models \psi \\ (\sigma, t) &\models \varphi \mathcal{U}_{[a,b]} \psi \quad \text{iff } \exists t' \in [t+a, t+b] : (\sigma, t') \models \psi \text{ and} \\ &\quad \forall t'' \in [t, t'] : (\sigma, t'') \models \varphi \end{aligned}$$

For convenience we write $\sigma \models \varphi$ for $(\sigma, 0) \models \varphi$. Two additional temporal operators can be defined with $\mathcal{U}_{[a,b]}$, $\neg$, and $\top$: Eventually ($\Diamond_{[a,b]} \varphi := \top \mathcal{U}_{[a,b]} \varphi$) and Always ($\Box_{[a,b]} \varphi := \neg \Diamond_{[a,b]} \neg \varphi$).

This is the full STL logic, which our monitor cannot handle. The following is a restriction of STL to a fragment, DiSTL, that is tractably monitorable.

A general formula of this fragment will be labeled φ^τ .

Definition 1 (DiSTL). *DiSTL is a fragment of STL with syntax:*

$$\begin{aligned} \varphi^a &:= \top \mid \perp \mid p_n \mid \neg p_n \mid \varphi^a \vee \varphi^a \mid \varphi^a \wedge \varphi^a \\ \varphi^\tau &:= \varphi^a \mid \varphi^\tau \vee \varphi^\tau \mid \varphi^\tau \wedge \varphi^a \mid \Diamond_I \varphi^\tau \mid \Box_I \varphi^a \mid \varphi^a \mathcal{U}_I \varphi^\tau \end{aligned}$$

where $I = [a, b]$ is an interval such as that attached to the Until in the STL syntax.

In the semantics, $f_{p_n}(x_1(t), \dots, x_N(t)) = x_n(t) - \beta_n$ for some rational number β_n .

4 Inputs, Outputs, and Benchmark

The monitor takes $N+1$ inputs: N input signals and a formula. The input signals are passed in as pairs, (t, v) , where t is the time and v is the value at time t . The monitor then interpolates between these pairs piecewise-linearly, handling a continuous-time signal.

The formula is created using Rust types provided in the monitor – see ‘src/main.rs’ in the experiments.

The experiments provide a script to generate new input signals, and three different formulas are hardcoded.

As output, the monitor returns a set of boxes – these boxes, in $N + 1$ dimensions, represent satisfying system states. Every point in a box is a system state, where (when indexing by 0) the n th dimension of a point is agent n ’s local time. The 0th dimension is the *reference time*, a theoretical global time for that system state.

The benchmark returns runtime information of the monitor, separately for each of the three hardcoded formulas, when 1) varying the number of agents N (fixing $\varepsilon = 0.05$) and 2) when varying the constant ε (fixing $N = 4$). The frequency of the signal is set to 10/sec, the number of times the signal crosses between positive and negative. Results can be found in the ‘results’ directory. The generated agent signals are 601 samples in length. The hardcoded formulas are the following:

1. $p_1 \wedge p_2 \wedge \dots \wedge p_N$
2. $\Diamond_{[1,2]} \Box_{[0,3]} (p_1 \vee p_2 \vee \dots \vee p_N)$
3. $p_1 \mathcal{U}_{[0,3]} (\Box_{[0,6]} (p_2 \vee p_3 \vee \dots \vee p_N))$.

These formulas were designed to handle arbitrary numbers of agents ≥ 2 .

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: International Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems. pp. 152–166. Springer (2004)
2. Mills, D., Martin, J., Burbank, J., Kasch, W.: Network time protocol version 4: Protocol and algorithms specification. Tech. rep., Internet Engineering Task Force (2010)